In [1]:
```
import tensorflow as tf
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
from IPython.display import HTML
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import warnings

import tensorflow as tf

print("Num GPUs Available: ", len(tf.config.experimental.list_physical_devic
```

```
Num GPUs Available:  0
```

In [2]:
```
IMAGE_H = 256
IMAGE_W = 256
BATCH_SIZE = 32
CHANNELS =3


train_datagen = ImageDataGenerator (
    rescale=1./255,
    rotation_range = 40,
    width_shift_range = 0.2,
    height_shift_range = 0.2,
    shear_range = 0.2,
    zoom_range = 0.2
)

train_generator =train_datagen.flow_from_directory(
    'banana_disease_dataset/train',
    target_size=(IMAGE_H,IMAGE_W),
    class_mode='sparse',


)
```

```
Found 1289 images belonging to 7 classes.
```

In [3]:
```
validation_datagen =ImageDataGenerator(
    rescale=1./255
)

validation_generator =validation_datagen.flow_from_directory(
    'banana_disease_dataset/val',
    target_size=(IMAGE_H,IMAGE_W),
    class_mode='sparse',

)
```

```
Found 369 images belonging to 7 classes.
```

In [4]:
```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "banana_disease_dataset/train",
    shuffle = True,
    image_size = (IMAGE_H,IMAGE_W),
)
```

```
Found 1289 files belonging to 7 classes.
```

In [5]:
```
class_names = dataset.class_names
class_names
```

Out[5]:
```
['BACTERIAL SOFT ROT',
 'BANANA LEAF RUST',
 'BLACK SIGATOKA',
 'BUNCHY TOP VIRUS',
 'CORDANA LEAF SPOT',
 'PANAMA DISEASE',
 'PSEUDOSTEM WEEVIL']
```

In [6]:
```
test_datagen =ImageDataGenerator(
    rescale=1./255
)

test_generator =test_datagen.flow_from_directory(
    'banana_disease_dataset/test',
    target_size=(IMAGE_H,IMAGE_W),
    class_mode='sparse',

)
```

```
Found 190 images belonging to 7 classes.
```

In [7]:
```
input_shape = (IMAGE_H, IMAGE_W, CHANNELS)
n_classes = 7

model = tf.keras.models.Sequential([


    tf.keras.layers.Conv2D(16,(3,3),1,activation='relu',input_shape =input_s
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Conv2D(32,(3,3),1, activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Conv2D(64,(3,3),1, activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(n_classes, activation='softmax')

])
```

In [8]:
```
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 254, 254, 16)      448

 max_pooling2d (MaxPooling2  (None, 127, 127, 16)      0
 D)

 conv2d_1 (Conv2D)           (None, 125, 125, 32)      4640

 max_pooling2d_1 (MaxPoolin  (None, 62, 62, 32)        0
 g2D)

 conv2d_2 (Conv2D)           (None, 60, 60, 64)        18496

 max_pooling2d_2 (MaxPoolin  (None, 30, 30, 64)        0
 g2D)

 flatten (Flatten)           (None, 57600)             0

 dense (Dense)               (None, 256)               14745856

 dense_1 (Dense)             (None, 7)                 1799

=================================================================
Total params: 14771239 (56.35 MB)
Trainable params: 14771239 (56.35 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [9]:
```
model.compile(
    optimizer ='adam',
    loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits =False),
    metrics=['accuracy']
)
```

In [10]:
```
299/32
```

Out[10]:
```
9.34375
```

In [11]:
```
model.fit(
    train_generator,
    validation_data = validation_generator,
    verbose = 1,
    shuffle = True,
    epochs = 50
)
```

```
Epoch 45/50
41/41 [==============================] - 94s 2s/step - loss: 0.1862 - ac
curacy: 0.9286 - val_loss: 0.4005 - val_accuracy: 0.8916
Epoch 46/50
41/41 [==============================] - 91s 2s/step - loss: 0.1447 - ac
curacy: 0.9465 - val_loss: 0.3716 - val_accuracy: 0.9051
Epoch 47/50
41/41 [==============================] - 96s 2s/step - loss: 0.1328 - ac
curacy: 0.9550 - val_loss: 0.3578 - val_accuracy: 0.9133
Epoch 48/50
41/41 [==============================] - 94s 2s/step - loss: 0.2068 - ac
curacy: 0.9185 - val_loss: 0.4249 - val_accuracy: 0.9051
Epoch 49/50
41/41 [==============================] - 91s 2s/step - loss: 0.1464 - ac
curacy: 0.9372 - val_loss: 0.3617 - val_accuracy: 0.9051
Epoch 50/50
41/41 [==============================] - 91s 2s/step - loss: 0.1062 - ac
curacy: 0.9589 - val_loss: 0.3686 - val_accuracy: 0.9106
```

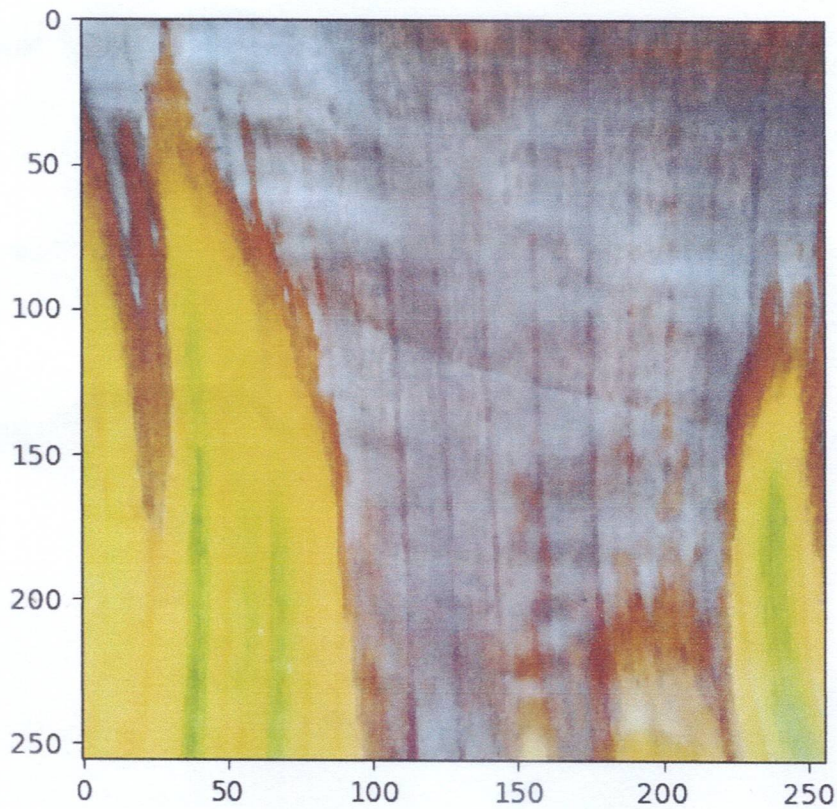
Out[11]:
```
<keras.src.callbacks.History at 0x1cf994c7a30>
```

In [12]:
```
scores = model.evaluate(test_generator)
```

```
6/6 [==============================] - 6s 932ms/step - loss: 0.3633 - accu
racy: 0.9211
```

In [13]:
```
import numpy as np

for images_batch, labels_batch in test_generator:

    first_images = images_batch[0]
    first_label = int(labels_batch[0])

    print("first image to predict")
    plt.imshow(first_images)
    print("actual label: ", class_names[first_label])

    batch_prediction = model.predict(images_batch)
    print("predicted label:",class_names[np.argmax(batch_prediction[0])])

    break
```

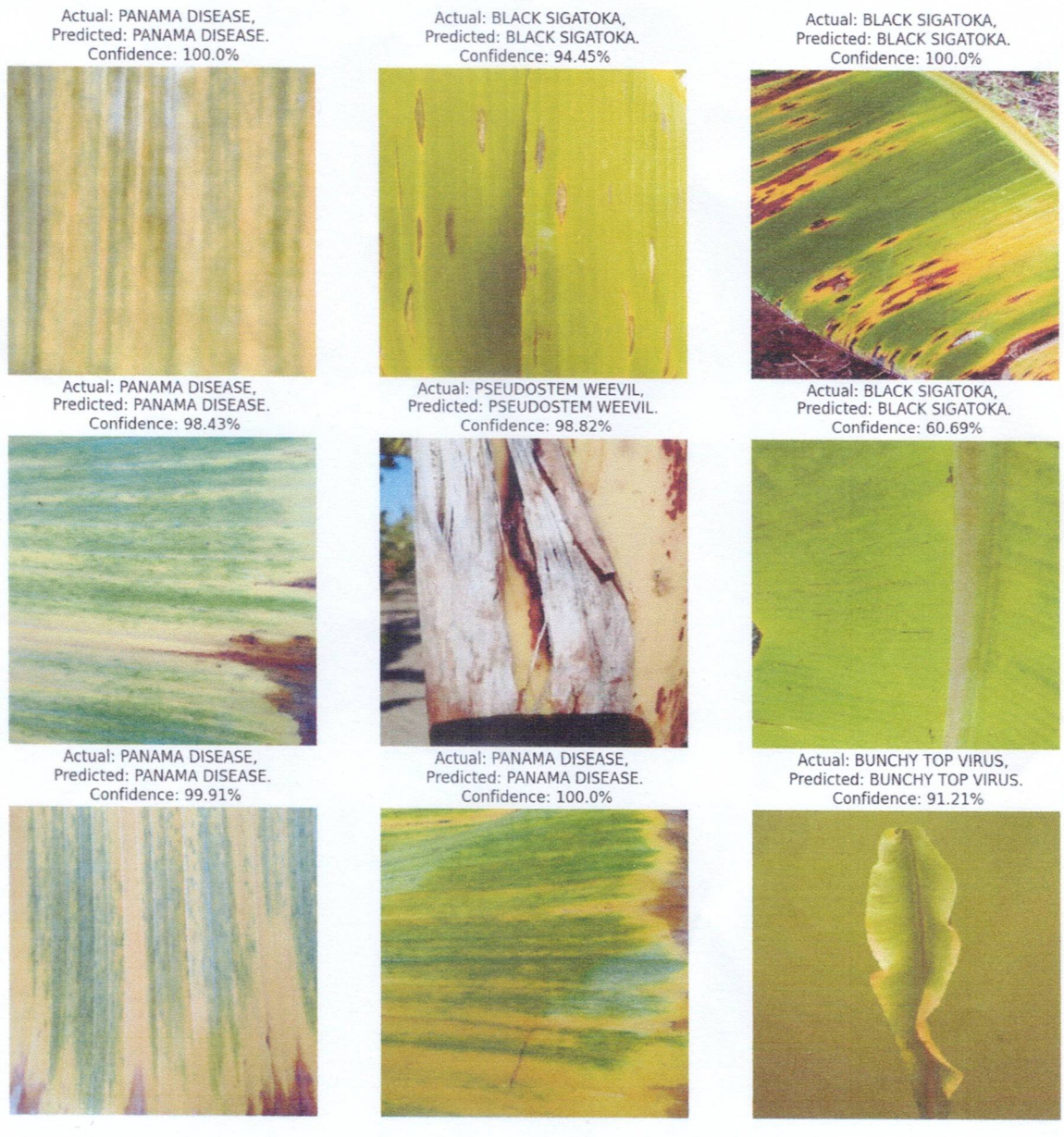
```
first image to predict
actual label:  PANAMA DISEASE
1/1 [==============================] - 1s 649ms/step
predicted label: PANAMA DISEASE
```



In [14]:
```
def predict(model, img):
    img_array = tf.keras.preprocessing.image.img_to_array(images[i])
    img_array = tf.expand_dims(img_array,0)

    predictions = model.predict(img_array)

    predicted_class = class_names[np.argmax(predictions[0])]
    confidence = round(100 * (np.max(predictions[0])), 2)
    return predicted_class, confidence
```

In [*]:
```
plt.figure(figsize=(15,15))
for images, labels in test_generator:
    for i in range(9):
        ax =plt.subplot(3, 3, i + 1)
        plt.imshow(images[i])

        predicted_class, confidence = predict(model, images[i])
        actual_class = class_names[int(labels[i])]

        plt.title(f"Actual: {actual_class}, \n Predicted: {predicted_class}.
        plt.axis("off")
    break
```

```
1/1 [==============================] - 0s 189ms/step
1/1 [==============================] - 0s 80ms/step
1/1 [==============================] - 0s 83ms/step
1/1 [==============================] - 0s 78ms/step
1/1 [==============================] - 0s 67ms/step
1/1 [==============================] - 0s 74ms/step
1/1 [==============================] - 0s 70ms/step
1/1 [==============================] - 0s 74ms/step
1/1 [==============================] - 0s 74ms/step
```



In [*]:
```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model =converter.convert()

with open("banana_diseases_model.tflite", 'wb') as f:
    f.write(tflite_model)
```

```
INFO:tensorflow:Assets written to: C:\Users\USER1~1\AppData\Local\Temp\tmp
mn9jex3b\assets

INFO:tensorflow:Assets written to: C:\Users\USER1~1\AppData\Local\Temp\tmp
mn9jex3b\assets
```